

KernelAscent: The Complete Report

Three weeks of recursive self-improvement experiments in GPU-kernel optimization —
results, mechanism, and intuition

KernelAscent

September 18, 2026

Abstract

KernelAscent asks a question that raw capability benchmarks cannot: not *how good* a model is at writing GPU kernels, but whether it can *make itself better* at writing them — and whether that improvement *compounds*. We built a five-rung ladder of recursion (one-shot capability; weight-level self-training; frozen-weight procedure self-editing; a closed model rewriting an open trainee’s harness; and open-ended self-play where a model authors the very frontier it must then beat) and ran it for three weeks on a 72-GPU cluster across model scales from 0.5B to 14B and a panel of frontier API models. This report is the full synthesis: the design rationale for each rung, every quantitative result with its uncertainty, the mechanistic account of *why* verified self-training does not compound in this domain, and the accumulated intuition and operational lessons of the run. Our headline is a rigorously-powered null: at matched compute, carrying a self-training lineage forward does not beat a matched reset (a statistically equivalent result, $BF_{01} \approx 14.7$), while verified *search* at the same budget does better — localizing the failure to a discovery→assimilation bottleneck rather than to an absence of capability.

Contents

1 Thesis: recursion, not capability	3
2 The RSI ladder: five rungs of recursion	3
3 Infrastructure: three weeks on 72 GPUs	4
4 Results by rung	5
4.1 T1 — Capability	5
4.2 T2 — Weight-RSI: a rigorously-powered compounding null	5
4.3 T3 — Procedure-RSI	6
4.4 T4 — Closed→Open	6
4.5 T5 — Self-play (open and closed)	7
5 Cross-cutting analyses: the mechanism	7
5.1 Coverage vs. scale, and the “correctness wall” as a sampling artifact	7
5.2 Zero-score forensics: the wall is a formation failure	8
5.3 Search beats training at matched compute	8
5.4 WHY-RSI: the weight-level mechanism and the inverted-U in scale	8
5.5 Probe-as-intervention: correctness is represented better than it is generated	9

6	Synthesis: why self-improvement does not compound here	9
6.1	Why self-improvement does not compound	9
7	The accumulated intuition (what three weeks taught us)	11
8	What would break the plateau: predictions	12
9	Limitations and scope	12
10	Conclusion	12

1 Thesis: recursion, not capability

The central intellectual commitment of KernelAscent is that the interesting axis of “self-improvement” is **recursion**, not the raw height of a single-shot score. A model that writes an excellent kernel once has demonstrated capability; a model whose *next* kernel is better *because of* the last one has demonstrated something qualitatively different. Most benchmarks conflate the two because they measure a fixed model against a fixed test. We instead measure the *derivative*: does an improvement, once made, become part of the improver, so that improvements accumulate?

GPU-kernel optimization is an unusually clean substrate for this question. Correctness is checkable against an fp32 reference with no ambiguity; speed is measurable against a roofline; the task space is procedurally generable at controlled difficulty; and there is a genuine, unbounded frontier (there is always a faster kernel). A verifier this crisp removes the usual confound of a noisy reward, so any failure to compound cannot be blamed on the grader.

Intuition. *If self-improvement is real anywhere, it should be visible where the reward is a clean, programmatic verifier and the frontier is real. Kernels are that place. So a null here is informative in a way a null on a fuzzy task would not be: it cannot be explained away by reward noise.*

2 The RSI ladder: five rungs of recursion

Each rung isolates a different *channel* through which improvement could feed back into the improver, in increasing order of recursion depth. The design principle throughout is the *matched control*: every “recursive” arm is paired with a non-recursive arm at identical compute, so a positive result cannot be attributed to simply spending more.

T1 — Capability (one shot). Can the model write a correct, fast kernel in a single attempt? No recursion; this is the baseline skill that gates everything above it. Metric: correct rate, fast rate, mean bounded attainment C .

T2 — Weight-RSI (the title claim). Across rounds, the model generates kernels, keeps the verified-correct ones, and trains its own weights on them, then is re-scored on a held-out split. Because the weights change, the *improver itself* changes. The decisive control is **lineage vs. matched reset**: does carrying the accumulated weights forward beat restarting from base each round at equal compute? Metric: $L - R$ per round. Open-weight only.

T3 — Procedure-RSI (recursion without weights). A frozen-weight model edits its own *executable research procedure* — a strategy library, a solver prompt, and an archive of verified kernels — from the evidence of the last round. This is how a closed API model can sit on the RSI axis at all. Metric: held-out procedure quality Q , gain vs. the frozen procedure.

T4 — Closed→Open (improvement through tooling). A closed frontier “researcher” rewrites the *training harness* (data selection, LoRA hyperparameters, curriculum) of an open-weight trainee. The closed model never changes its own weights; the improvement is transmitted through the tooling it authors. Metric: open-trainee C under the improved vs. frozen harness.

T5 — Self-play (the true-RSI metric). The model *authors its own strictly-harder tasks* and solves-and-improves on them, so difficulty and capability co-evolve. To separate “a harder curriculum” from “a co-evolving author” we run three arms at equal budget on one fixed held-out ladder: **S** (static frontier), **F** (escalating frontier, frozen author), **L** (escalating frontier,

live/evolving author). The primary signal is $L - F$ — the extra gain from letting the author evolve — with anti-reward-hacking gates rejecting degenerate self-authored tasks.

Intuition. *The ladder is deliberately ordered by how “expensive” the feedback channel is to fake. A one-shot score (T1) says nothing about recursion. A weight update (T2) is the strongest possible feedback — it modifies the model itself — so it is the title claim. Procedure editing (T3) and harness authoring (T4) are weaker, externalized channels that a frozen model can still use. Self-play (T5) is the only rung where the problem distribution itself is endogenous, which is the real definition of open-ended improvement. We expected the strongest evidence at T2 and the most fragile at T5; the data inverted our expectation about where gains would even appear.*

3 Infrastructure: three weeks on 72 GPUs

The experiments ran on a 9-node, 72-GPU cluster (A100-class), continuously for three weeks. A few infrastructure decisions shaped what was learnable, and several operational lessons are worth recording because they materially affected the results.

Compute discipline. Every rung’s recursive arm is compute-matched to its control by a full ledger that counts generation *and* verification *and* training tokens/steps. Without this, “self-training helped” is unfalsifiable — it may just be more sampling. The matched ledger is what turns T2 from a demo into a test.

Resumability. All runs checkpoint to S3 on a 300s daemon so an instance death never loses more than one interval, and pods push independently of the operator’s credentials. This is what made a three-week run survive tunnel drops, credential expiry, and node churn.

Operational lessons (recorded because they bit us).

- **Grading dominates wall-clock.** Crash-isolated per-kernel grading (each candidate graded in its own process so a segfaulting kernel cannot take down the harness) costs ~ 12 s/kernel. For coverage-style experiments requiring hundreds of grades per evaluation, a single round can exceed 2.5 hours — which made the randomized coverage-transplant experiment *computationally infeasible* and forced us to establish the coverage mechanism more cheaply from p -maps and forensics instead.
- **Large-model rounds are slow.** A 7B compounding round costs ~ 64 minutes. At six rounds per seed that is ~ 6.4 hours per seed, so the large-scale arms accumulate statistical power slowly and are sensitive to any run reset from node churn — the reason our 7B/14B equivalence is pooled-significant but not yet individually powered.
- **Measurement hygiene is a result-integrity issue.** We twice caught inflated counts caused by stale local caches masking a silently-failed sync, and once caught a “fleet collapse” that was merely a dropped port-forward tunnel hiding two dozen busy GPUs. Every headline number in this report is taken from a *verified* re-read of canonical storage, not a convenience cache.
- **A single silent bug can fabricate a finding.** An early “compounding collapse” (both train and held-out C dropping to exactly 0.000 after one SFT step) was a generation-after-SFT bug, not a scientific null; a zero-update adapter evaluated correctly, which is how we distinguished

the two. Pre-fix runs are therefore *uninterpretable* and are excluded from every statistic by a canonical-seed allowlist.

Intuition. *The most dangerous failure mode in an autonomous, long-horizon run is not a crash — it is a plausible wrong number. Crashes announce themselves; a stale cache or a contaminated seed quietly shifts an estimate. The single highest-leverage habit of the whole run was refusing to trust any number that had not been re-derived from canonical storage with the broken seeds excluded.*

4 Results by rung

4.1 T1 — Capability

On the E0 gate (15 fixed Medium procedural tasks, $k=5$, 75 trials/model, 95% Wilson CIs), capability separates into *two walls*: correctness ranks the low/mid band, while *speed* (**fast_rate**, the fraction beating the `min(eager, torch.compile)` roofline by $\geq 1.10\times$) is the frontier discriminator.

Model	correct	fast	mean C
GPT-5.6 Terra (api)	0.920	0.667	0.793
GPT-5.6 Sol (api)	0.907	0.600	0.753
Kimi K2.5	0.387	0.240	0.313
Fable 5.1	0.360	0.227	0.293
Palmyra-X5	0.533	0.040	0.287
gpt-oss-120b	0.467	0.080	0.273
Qwen2.5-Coder-7B (open)	0.280	0.000	0.140

Table 1: T1 capability (top rows). *Every* open model $\leq 14\text{B}$ has **fast_rate** = 0: they can sometimes write a correct kernel but essentially never a frontier-fast one; Coder-14B and Gemma-3-27B additionally hallucinate `torch` internals. Source: `leaderboard.json`.

Intuition. *The two walls matter for everything above: T1 shows that below the frontier, models are correctness-limited long before they are speed-limited. So the RSI rungs are mostly testing whether a model can compound its way up the correctness wall, not whether it can shave microseconds — which is why “where C moves, it moves via correctness acquisition, not speed” recurs throughout.*

4.2 T2 — Weight-RSI: a rigorously-powered compounding null

The title claim is where the strongest evidence lives, and it is a *null*. Over multi-round rejection-sampling SFT, carrying the lineage forward does not beat a matched reset.

Finding. Pooled across scales (0.5B–14B, $n=229$ round-comparisons), $L-R = -0.002 [-0.017, +0.013]$, TOST-equivalent at $\delta=0.05$ with $\text{BF}_{01} \approx 14.7$ — *strong* evidence for the null. The 1.5B and 3B scales are individually equivalent (BF_{01} 7.7, 8.1); 0.5B is marginal and negative if anything; 7B/14B are still accumulating (~ 64 min/round) but their early points fall inside the equivalence bounds.

Scale	n	$L-R$ (TOST 90% CI)	BF_{01}
Qwen2.5-Coder-0.5B	51	$-0.018 [-0.052, +0.016]$	4.8 (marginal)
Qwen2.5-Coder-1.5B	93	$+0.009 [-0.013, +0.031]$	7.7 (EQUIV)
Qwen2.5-Coder-3B	78	$-0.007 [-0.034, +0.020]$	8.1 (EQUIV)
Qwen2.5-Coder-7B	5	$+0.024 [-0.065, +0.113]$	2.2 (underpowered)
Pooled	229	$-0.002 [-0.017, +0.013]$	14.7 (EQUIV)

Table 2: Lineage vs. matched reset, per scale and pooled (`equivalence_tost.py`). Per-round, $L-R$ shows a *transient* early advantage (rounds 1–2, e.g. Qwen-3B/1.5B $+0.10$) that **collapses to ≈ 0 by round 3** — so accumulated self-training does not beat a fresh reset averaged over rounds.

The uncontrolled T2 boards (`rsi_leaderboard.json`, `tier_speed_rsi.json`) do show apparent per-model “compounding” — the strongest being **stable-code-3b**, self-fresh-frozen $+0.226$, sign-consistent across 2 seeds; and starcoder2-15b $+0.074$ across 3 seeds. But the decisive recursion-interruption fork (A1, `fork.json`) dissolves it: for deepseek-1.3b, continuing to self-train vs. freezing the producer at the fork gives self-ckpt = -0.001 — a **one-time upgrade, not compounding**. Where C does move it is almost entirely *correctness acquisition*, not speed (`decomp.json`: deepseek-1.3b $\Delta_{\text{correct}} + 0.89$ vs $\Delta_{\text{speed}} + 0.38$).

Intuition. *The lineage-vs-reset null is the cleanest single fact in the project, and it is stronger than “updates are small.” If updates were merely small, lineage and reset would both drift slowly but lineage would still inch ahead. Instead they are indistinguishable — which means the accumulated weight state carries no useful cumulative information beyond what one round of fresh self-training recovers. That is a statement about the state, not the step size.*

4.3 T3 — Procedure-RSI

When a frozen-weight model rewrites its own executable procedure, frontier models improve it substantially (`trackc.json`): GPT-6 Astra Q $0.298 \rightarrow 0.854$ (**$+0.556$** , growing over 6 rounds), Claude Sonnet 5 **$+0.401$** , Mistral Large 3 **$+0.310$** ; a frozen-procedure control (Fable-5.1) sits at $+0.009$. But the self-modify mechanism (`selfplay_mech.json`) shows the gain is **68% realized at round 0** (one-shot), **75% non-recursive**, with the archive saturating by round 1–2 and strategy sets already diverse (Jaccard 0.12).

Finding. The only models with *sustained* per-round procedure gain (Astra, Mistral) are the ones that started from *low* Q_0 — i.e. they are consuming headroom, not demonstrating recursive self-insight. DeepSeek-V3.2’s -0.234 “self-degrade” is a 1-round interrupted-run artifact and should not be read as a finding.

4.4 T4 — Closed→Open

A closed researcher rewriting an open trainee’s training harness can help substantially (`combined_rsi.json`): Fable-5.1 $\rightarrow$ Qwen-1.5B reaches improved-frozen **$+0.472$** (peak 0.472, C $0.147 \rightarrow 0.501$ vs frozen 0.029), and Fable-5.1 $\rightarrow$ Qwen-7B **$+0.313$** . But the sign is *pair-dependent*: the same researcher *hurts* other trainees (Fable→OpenCoder-1.5B -0.265 , Fable→DeepSeek-1.3B -0.444), and many researcher rows are flat. As with T4’s cousin T2, an open question (ROADMAP A4) is whether the $+0.472$ is correctness rather than speed.

Intuition. *Improvement does transmit through authored tooling — a closed model that cannot change its own weights can still cause a better open model. But it is brittle and trainee-specific,*

which reads less like “the researcher discovered a better training recipe” and more like “it happened to match this trainee’s correctness headroom.” Transmitted improvement is real but not yet a reliable, general channel.

4.5 T5 — Self-play (open and closed)

The primary signal is $L-F$ (extra gain from a co-evolving author beyond an escalating curriculum). **Open** (`selfplay.json`, 24 runs): most models $\leq 15\text{B}$ show $L-F \approx 0$ (adaptive curriculum only). The clearest positive is starcoder2-15b $L-F = +0.232$ — but on only *one* accepted model-proposed task, so it is suggestive at best; deepseek-coder-6.7b-base reaches $+0.154$ (3 proposed). **Closed** (`selfplay_closed.json`, procedure channel, 11 runs): *every* row is $L-F = 0.0$ except Mistral Large 3 at -0.584 , even though GPT-5.6 Sol/Terra and Sonnet-5 each proposed 36–37 tasks. **Open-ended** (`open_rsi.json`, 16 runs): *all* verdicts are “one-time (open = fixed)”.

Finding. The self-play diagnosis (`selfplay_diag.json`) explains the near-universal $L-F \approx 0$: the learned author suffers an **authored-task solve-rate collapse** — proposed tasks are either trivial (solver $\geq 90\%$) or unsolvable ($\leq 10\%$), rarely in the learnable 20–80% band — and the “goldilocks” rescue did not fire. Open-endedness is a one-time upgrade here, not a compounding engine, because the author cannot reliably manufacture in-band difficulty.

(Note: an earlier README listed closed Sonnet-5 $+0.045$ / GPT-5.6-Sol $+0.041$ as “emerging positives”; these are *not* in the current data and are treated as stale.)

5 Cross-cutting analyses: the mechanism

The rung-by-rung results tell us *whether* improvement compounds. The following analyses tell us *why* it does not, and together they form the mechanistic core of the report.

5.1 Coverage vs. scale, and the “correctness wall” as a sampling artifact

Per-task p -maps on a fixed 29-task bank (`pmap_curve.json`) trace coverage (tasks with ≥ 1 verified-correct sample) from 13.8% at 0.5B to 86.2% at 14B, and show that below the wall $\text{pass}@K$ dwarfs $\text{pass}@1$:

Size	K	coverage	$\text{pass}@1$	$\text{pass}@K$	K -ratio
0.5B	64	13.8%	0.009	0.109	$\sim 12\times$
1.5B	64	27.6%	0.014	0.235	$\sim 17\times$
3B	64	82.8%	0.112	0.777	$\sim 7\times$
7B	48	82.8%	0.079	0.799	$\sim 10\times$
14B	24	86.2%	0.655	0.862	$\sim 1\times$

Table 3: Coverage and sharpening across scale. The sharpening geometry (`results_auto.tex`): the *sharpenable* band ($0 < p < 0.2$) is 14%/28%/59%/79%/0% at 0.5/1.5/3/7/14B — it peaks mid-scale and collapses at 14B, where all mass is already *reliable* ($p \geq 0.2$).

Intuition. The sub-2B “correctness wall” is not an inability — it is a low per-sample success probability. Give a small model enough draws and correct kernels appear ($\text{pass}@K \gg \text{pass}@1$). This reframes the entire problem: self-training does not need to learn to be correct, it needs to

reliably reproduce a correctness that is already latent in the sampling tail. That is a sharpening problem, not an exploration problem.

5.2 Zero-score forensics: the wall is a formation failure

Classifying *why* each 0-score candidate failed (`forensics_summary.json`) shows the sub-3B wall is a **kernel-formation** wall (no parseable ModelNew), not runnable-but-wrong code: `no_extract` dominates at 0.5B (72%), DeepSeek-1.3B (90%), Yi-1.5B (96%). The failure *locus marches downstream with scale*: incoherence $\rightarrow$ truncation/syntax (Qwen-7B: 75% syntax) $\rightarrow$ API-hallucination $\rightarrow$ wrong-output $\rightarrow$ correct (Qwen-14B: 65% correct, only 0.2% `no_extract`).

Intuition. *Small models do not fail by writing subtly-wrong kernels — they fail by not producing a well-formed kernel at all. That is why coverage is so low and why self-training starves: there is nothing correct to train on. As scale rises the bottleneck migrates from “can it write a kernel” to “is the kernel right” to “is it fast.”*

5.3 Search beats training at matched compute

Pooled over 83 round-comparisons (`search_vs_train.json`), lineage—bestof $N = -0.113 [-0.153, -0.074]$ (CI excludes zero), and verified search wins 69.9% of rounds; every scale is individually negative (0.5B -0.140 , 1.5B -0.080 , 3B -0.145). A separate matched-budget baseline sweep (`baselines.json`, budget 20) is weaker and mixed — Qwen-1.5B recursion.gain $+0.055$ over its best non-recursive baseline, but starcoder2-3b -0.073 — so the strong, consistent statement is the search-vs-lineage one.

Finding. At a matched compute ledger, best-of- N verified search beats lineage self-training (lineage—bestof $N = -0.113 [-0.153, -0.074]$, CI excludes zero). The useful solutions are *already reachable by sampling*; search exploits them directly, while training must compress them into shared weights and have the gain survive.

5.4 WHY-RSI: the weight-level mechanism and the inverted-U in scale

Across 133 weight-RSI runs (`mech_analysis.json`) the correctness wall is the primary gate: crossing rises from 54% below 2B to 92% at ≥ 2 B. Among the 97 wall-crossers, RSI is an **inverted-U in scale**:

Band	n	wall-cross	LoRA drift	retention	frac RSI
<2B	67	0.54	0.019	0.013	0.03
2–8B	45	0.98	0.246	0.180	0.49
≥ 9 B	21	0.81	0.726	0.339	0.19

Table 4: The inverted-U (`mech_analysis.json`, $n=133$). RSI peaks at 2–8B; the largest models drift *most* in weight space yet compound *least* — motion without progress near the roofline. Only 11/133 runs compounded (stable-code-3b recurs). When drift occurs it localizes to *late* layers (task adaptation) and the held-family transfer gap is ≈ 0 (no transferable meta-skill).

Finding. A subtle and important nuance: on this bank, compounders have *lower* generation diversity than flat runs (0.41 vs 0.52), not higher. The discriminator between compounding and

flat among wall-crossers is **sustained LoRA drift + retention** ($\sim 5\times$ higher in compounders), *not* diversity preservation. So “diversity collapse causes the null” is *not* supported by our weight-RSI data — compounding here looks like *productive convergence onto a good basin*. Diversity contraction remains a plausible *cross-round option-value* lock (see the Discussion) but we flag it as an association whose clean test is a diversity-preserving intervention, and we explicitly do *not* claim it as the mechanism.

Intuition. *Scale does not monotonically help RSI — it is an inverted-U. Small models cannot form correct kernels often enough to feed the training set (formation-starved). Mid-scale models sit on the widest learnable frontier and compound best. Large models cover almost everything already, so their weights churn (high LoRA drift) without moving the held-out score — “motion without progress” near the roofline. The binding constraint slides from formation (small) to headroom (large).*

5.5 Probe-as-intervention: correctness is represented better than it is generated

A linear probe reads “is this kernel correct” out of mid-to-late layers at high AUC (Qwen-3B best_auc **0.985** at depth 0.86, while its actual correctness rate is 0.105) — the model *represents* correctness far better than it *generates* it (`interp.json`). Using the probe to select at equal budget (`probe.intervene.json`) shows large lifts on some checkpoints (Qwen-0.5B PROBE–RANDOM +0.42, Qwen-3B +0.40) but negatives on others (Qwen-7B −0.27) and noise across repeated draws on the small bank.

Finding. We report this carefully. The high *pooled* AUC partly reflects the probe separating easy tasks from hard ones, *not* ranking candidates within a task; the within-task ranking signal is only modest (≈ 0.67) and there is **no clean matched-budget harvesting advantage** yet. The defensible claim is qualitative — correctness is internally decodable well above generation success — which is the discovery→assimilation gap seen from the inside, not a deployable self-verification win. A leave-task-out, within-task ranking evaluation on a larger task bank is the needed follow-up.

Intuition. *A linear probe can read “is this kernel correct” out of mid-to-late layers at high AUC, yet the model emits a correct kernel far less often than it can internally recognize one. The model knows more than it generates. This is the same discovery→assimilation gap seen from the inside: the limiting step is turning latent competence into a reliably-produced output.*

6 Synthesis: why self-improvement does not compound here

6.1 Why self-improvement does not compound

The mechanism: a discovery→assimilation bottleneck (sharpening, not exploration). Our two strongest results jointly localise the failure. Best-of- N search beating lineage training at matched compute (lineage–bestof $N = -0.113 [-0.153, -0.074]$), together with the large pass@ $K \gg$ pass@1 gap, shows that *useful solutions are already present in the sampling tail* but are not reliably produced in one sample. The limiting pipeline is therefore

rare verified sample $\rightarrow$ selected SFT example $\rightarrow$ weight update $\rightarrow$ reliably reusable gain,

and it is *lossy*: search can exploit a rare discovery directly, whereas training must compress it into a shared parameterisation, preserve other skills, and have the gain survive later rounds. The

lineage-vs-reset null (inheriting updates does not beat restarting) is direct evidence *against a useful cumulative learning state*, not merely evidence that individual updates are small. This is the classic self-training signature: selected-sample training concentrates probability on *already-reachable* successes faster than it expands what is reachable (cf. STaR, ReST, ReST^{EM}, expert iteration; and the RLVR capability-boundary debate, Yue et al. 2025).

A formal fixed point — with its limits stated precisely. Under an *idealised* support-preserving rejection-conditioning operator R (unlimited accepted samples, exact per-task fitting, no cross-task transfer), exact solution-support coverage $C_0(\pi) = \Pr_x[p_\pi(x) > 0]$ is invariant and R is idempotent after one step: $C_0(R\pi) = C_0(\pi)$ and $R^2\pi = R\pi$. On tasks with positive success probability R places all mass on accepted outputs; on $p=0$ tasks no accepted distribution exists. So an idealised verified-SFT loop is a *one-step fixed point in exact coverage*. **We are careful about what this does *not* say.** (i) Neural SFT is not this operator — shared parameters can generalise and raise the probability of unsampled outputs, so real SFT need not preserve support. (ii) *Finite-budget* coverage is *not* invariant: $C_K(\pi) = \mathbb{E}_x[1 - (1 - p_\pi(x))^K]$ rises sharply under sharpening even when C_0 is fixed — “pass@ K coverage is a fixed point” is false. (iii) Zero observed successes only bound $p \lesssim 3/K$; our “unreachable mass” is more precisely *undiscovered mass at the evaluated budget*. (iv) A training-operator fixed point is *not* a parameter-space local optimum. We therefore report an **operational self-training plateau / protocol-relative fixed point**, never an unqualified “local optimum.”

Why verified search escapes where training does not. Search does not escape the policy’s support; it escapes the *one-sample* bottleneck and avoids the assimilation bottleneck. For success probability p , $P(\text{hit in } N) = 1 - (1 - p)^N$ makes extra draws valuable when p is small, and the verifier keeps the winner. Search (a) uses a discovery directly rather than compressing it into weights, (b) allocates candidates per-task rather than via one shared update, and (c) does not permanently contract the proposal policy. The precise statement: *verified search accesses useful existing tail capability more efficiently than the tested training loop consolidates it*. (This does not settle lifetime economics — a training update could amortise over enough future queries; that needs a separate deployment-horizon study.)

Diversity contraction as a reinforcing lock (associational). Selected-sample SFT can drive a loop: current common winners $\rightarrow$ accepted set $\rightarrow$ higher probability on those winners $\rightarrow$ fewer alternatives in the next accepted set. The loss is *option value* — rare decompositions and kernel families that would enable *later* discoveries. If undiscovered useful mass U_t shrinks, later rounds find fewer novel successes $(1 - (1 - U_t)^N \leq NU_t)$ even as pass@1 rises on familiar ones. Higher retention among compounders and diversity collapse at large scale are *consistent* with this lock, but we flag them as association, not proof: low diversity can also reflect successful specialisation, and the cleanest test is a matched-reward, matched-example, diversity-*preserving* intervention.

The inverted-U, mechanistically. Partition tasks into *undiscovered* ($p \approx 0$ at budget), *learnable frontier* ($0 < p < 0.2$), and *near-ceiling* (p high). Most self-training value comes from the middle band. Small models: formation failures ($P(A | x) = P(F | x)P(A | F, x)$ with tiny $P(F)$) starve the accepted set; mid-scale: the frontier is widest, so discoveries become signal (RSI peaks); large scale: high coverage but little remaining headroom, so drift redistributes mass within already-covered behaviour (“churn without gain” — supported descriptively; “only superficial late-layer edits” is *not* yet established, as parameter distance is not calibrated functional change).

What would break the plateau, and what our data predict. Two distinct breakouts are needed: *accumulation* (retain/express more already-discoverable successes — could yield lineage>reset) and *frontier expansion* (make new strategies operationally reachable — required once the frontier is exhausted). A productive loop needs new discoveries + assimilation + retention + headroom simultaneously. The single most informative next experiment is a **coverage-injection** $\times$ **diversity-preserving** factorial: inject stronger-model / repair / search-generated solutions on *undiscovered* tasks and test whether inherited gains then emerge — separating “lack of discoveries” from “loss of discoveries after learning.” Exploration bonuses, novelty-selected accepted sets, prerequisite curricula, off-policy replay, and search-amplified expert iteration are ranked candidates; each helps only the specific deficit it targets (e.g. entropy cannot fix formation; off-policy replay of redundant winners adds no frontier information).

Scope — claims we do *not* make. We do not claim the models’ reachable set is mathematically invariant, that the weights sit at a genuine local optimum, or that RL/RLVR cannot expand capability. We claim, with strong evidence: under verified rejection-sampling SFT at the tested scales/budget, carrying the lineage forward does not beat a matched reset (equivalence, $\text{BF}_{01} \approx 14.5$); verified search is a better use of the same compute; and the binding internal signals are coverage/formation (small), a mid-scale learnable frontier, and drift-without-gain plus diversity contraction (large). The synthesis: *expert iteration without sufficient expert amplification, plus incomplete retention of the apprentice’s useful tail.*

7 The accumulated intuition (what three weeks taught us)

This section is the part that does not fit in a conference paper: the qualitative model we now carry in our heads about RSI in this domain.

1. **Self-training is a sharpening operator, not an exploration operator.** It concentrates probability on already-reachable successes faster than it expands what is reachable. Everything else follows from this.
2. **Coverage is the currency, and it is not self-generated.** A round of self-training can only reuse solutions the model already finds; it cannot manufacture coverage on tasks where the per-sample success probability is effectively zero at the evaluated budget. Compounding is coverage-limited, and coverage is exogenous to the loop.
3. **The bottleneck is discovery→assimilation, not knowledge.** The model represents correctness it cannot reliably generate (probe result), and search — which uses discoveries directly — beats training, which must assimilate them into weights and retain them. The lossy step is assimilation-and-retention.
4. **“Operational plateau,” not “local optimum.”** We are careful: the null is a protocol-relative fixed point of the tested verified-SFT loop, not a claim that the weights sit at a parameter-space local optimum. Finite-budget coverage is *not* invariant; a better loop could still move.
5. **Diversity contraction is a plausible reinforcing lock.** Selected-sample SFT can raise probability on current winners at the cost of the rare decompositions that would enable *later* discoveries. We flag this as association, not proof — the clean test is a diversity-preserving intervention.

6. **The open contribution is a better *harness*, not a better model.** Because RSI is a protocol we run, the way to break the plateau is to change the protocol: inject coverage on undiscovered tasks, preserve diversity in the accepted set, amplify the “expert” with search before assimilation. This is exactly what the public benchmark’s harness-submission track exists to surface.

8 What would break the plateau: predictions

Two distinct breakouts are needed and they target different deficits: *accumulation* (retain and express more already-discoverable successes — could make lineage finally beat reset) and *frontier expansion* (make genuinely new strategies operationally reachable — required once the current frontier is exhausted). A productive loop needs new discoveries + assimilation + retention + headroom *simultaneously*. Our single most informative next experiment is a **coverage-injection** × **diversity-preserving** factorial: inject stronger-model / repair / search-generated solutions on *undiscovered* tasks and test whether inherited gains then emerge — separating “lack of discoveries” from “loss of discoveries after learning.”

9 Limitations and scope

We claim, with strong evidence: under verified rejection-sampling SFT at the tested scales and budgets, carrying the lineage forward does not beat a matched reset; verified search is a better use of the same compute; and the binding internal signals are coverage/formation (small scale), a mid-scale learnable frontier, and drift-without-gain plus diversity contraction (large scale). We do *not* claim the reachable set is mathematically invariant, that the weights sit at a genuine local optimum, or that RL/RLVR (as opposed to rejection-sampling SFT) cannot expand capability. The pooled interval treats correlated rounds as independent, so per-trajectory (2–3 seed) uncertainty is the binding constraint; the coverage-transplant that would settle the mechanism directly proved computationally infeasible and is left to future work.

10 Conclusion

Three weeks of clean-verifier, compute-matched experiments across five rungs of recursion and scales from 0.5B to 14B converge on one picture: in GPU-kernel optimization, verified self-training *sharpens what a model already covers but does not explore*, so improvement does not compound — not because the models lack capability, but because the loop’s feedback channel assimilates rare discoveries lossily and cannot manufacture the coverage it would need to keep going. The result is a strong, honest null flanked by two positive findings (search beats training; the correctness wall is a sampling artifact) that together localize the failure and point at the intervention most likely to break it. KernelAscent ships that intervention as an open, automated submission track, so the community can try to turn the null into a curve.